

Promptathon — ICEPOT Prompt Makeover Challenge

Testleaf · Gen AI Training Program · ICEPOT Framework Assignment

Field	Details
Name	Gayathri
Role	Quality Analyst
Scenarios Chosen	03 — Shopping Cart, 04 — Role-Based Access Control, 05 — API Testing
AI Tool Used	Claude (Anthropic) — both the weak and the ICEPOT prompt were run in the same tool
Date	24 September 2026

Approach

- Chose 3 of the 7 scenarios and listed what is missing from each weak prompt.
- Filled the ICEPOT template (Instruction, Context, Example, Persona, Output Format, Tone) and combined it into one rewritten prompt.
- Ran the weak prompt first, then the ICEPOT prompt, in the same AI tool and saved both outputs.
- Compared both outputs side by side and recorded 3 observations per scenario.

Scenario 03 — Shopping Cart

Prompt Makeover Template

Template Field	My Entry
Scenario Name	Shopping Cart
Weak Prompt	"Test the cart functionality."
What I Feel is Missing	1. No application or domain (grocery, fashion, electronics?) and no platform (web/mobile). 2. No cart actions defined: add, remove, update quantity, and quantity limits. 3. No pricing rules: how subtotal, tax (GST) and delivery charges are calculated. 4. No coupon/discount rules: coupon codes, minimum order, maximum discount cap, expired coupons, one-coupon rule. 5. No out-of-stock handling or cart persistence (guest vs logged-in). 6. No output format, number of test cases or priority expected.
I – Instruction	Create end-to-end test cases for the shopping cart, validating add, remove and update-quantity actions, price calculations, coupon/discount application, out-of-stock handling and cart persistence.
C – Context	Online grocery web and mobile app. Quantity per item: 1–10. Subtotal = $\Sigma(\text{unit price} \times \text{qty})$. GST 5% is applied on the subtotal after discount. Delivery fee ₹40 when the discounted subtotal is below ₹500; free at ₹500 and above. Coupons: SAVE10 (10% off, max ₹100, min order ₹300), FLAT50 (₹50 off, min order ₹250), OLD20 (expired). Only one coupon per order; a coupon is auto-removed if the cart falls below its minimum. If an item goes out of stock while in the cart, show an "Out of stock" label and block checkout. Cart persists for logged-in users across sessions; a guest cart merges into the user cart on login.
E – Example	TC_CART_010 Coupon Apply SAVE10 on ₹1,500 order Steps: 1. Add items worth ₹1,500 2. Enter SAVE10 3. Click Apply Expected: Discount ₹100 (capped, not ₹150); GST 5% on ₹1,400 = ₹70; Delivery ₹0; Total ₹1,470 Priority: High Type: Positive
P – Persona	Act as an experienced E-commerce QA Analyst who specialises in pricing, calculation and checkout validation.
O – Output Format	A table with columns: Test Case ID, Module (Add/Update/Remove/Price/Coupon/Stock/Persistence), Scenario, Steps, Test Data, Expected Result (show the full calculation for price-related cases), Priority, Type. Minimum 15 test cases.
T – Tone	Clear, analytical and numbers-focused, so a junior tester can verify each value.

☒ Rewritten Prompt

You are an experienced E-commerce QA Analyst who specialises in pricing, calculation and checkout validation.

Task: Create end-to-end test cases for the shopping cart of an online grocery web and mobile app, covering add, remove and update-quantity actions, price calculations, coupons/discounts, out-of-stock handling and cart persistence.

Business rules: Quantity per item is 1–10. Subtotal = $\Sigma(\text{unit price} \times \text{qty})$. GST 5% is applied on the subtotal after discount. Delivery fee is ₹40 when the discounted subtotal is below ₹500 and free at ₹500 and above. Coupons: SAVE10 = 10% off, max ₹100, min order ₹300; FLAT50 = ₹50 off, min order ₹250; OLD20 = expired. Only one coupon

per order, and a coupon is auto-removed if the cart falls below its minimum. An item that goes out of stock while in the cart shows an "Out of stock" label and blocks checkout. The cart persists for logged-in users across sessions, and a guest cart merges into the user cart on login.

Example test case: TC_CART_010 | Coupon | Apply SAVE10 on ₹1,500 order | Steps: add items worth ₹1,500, enter SAVE10, click Apply | Expected: Discount ₹100 (capped, not ₹150); GST 5% on ₹1,400 = ₹70; Delivery ₹0; Total ₹1,470 | Priority: High | Type: Positive.

Output: A table with columns Test Case ID, Module, Scenario, Steps, Test Data, Expected Result (show the full calculation for every price-related case), Priority, Type. Minimum 15 test cases.

Tone: Clear, analytical and numbers-focused.

Output 1 — Weak Prompt ("Test the cart functionality.")

#	Test Case	Expected Result
1	Verify user can add a product to the cart	Product is added to the cart
2	Verify user can remove a product from the cart	Product is removed from the cart
3	Verify user can update the quantity of a product	Quantity is updated
4	Verify cart total is displayed correctly	Correct total is displayed
5	Verify cart icon shows the number of items	Cart count is updated
6	Verify empty cart message	"Your cart is empty" message is shown
7	Verify user can proceed to checkout	User is navigated to the checkout page
8	Verify cart items are retained after page refresh	Items remain in the cart

Result: 8 generic checks, no IDs, no test data, no priority, no domain rules.

Output 2 — ICEPOT Prompt

TC ID	Module	Scenario	Steps	Test Data	Expected Result	Priority	Type
TC_CART_001	Add	Add a single item to an empty cart	1. Open product 2. Click Add to Cart	Milk ₹60 × 1	Cart count = 1. Subtotal ₹60; GST 5% = ₹3.00; Delivery ₹40 (below ₹500); Total ₹103.00	High	Positive
TC_CART_002	Add	Add the same item twice	1. Add Milk 2. Add Milk again	Milk ₹60	Single line item with Qty 2 (no duplicate line). Subtotal ₹120	High	Positive
TC_CART_003	Update	Increase quantity to the maximum (10)	1. Add Rice 2. Click + until Qty 10	Rice ₹80 × 10	Qty 10 accepted, + button disabled. Subtotal ₹800; GST ₹40; Delivery ₹0; Total ₹840	High	Boundary
TC_CART_004	Update	Enter quantity above maximum	1. Type 11 in Qty field 2. Tab out	Qty = 11	Error "Maximum 10 units per item"; Qty stays 10; totals unchanged	High	Negative
TC_CART_005	Update	Decrease quantity to below minimum	1. Item at Qty 1 2. Click - / type 0	Qty = 0	- button disabled at Qty 1; typing 0 shows "Minimum quantity is 1"; item not removed silently	Medium	Boundary
TC_CART_006	Remove	Remove the last item from the cart	1. Cart has 1 item 2. Click Remove	Bread ₹45	Item removed; cart count 0; "Your cart is empty" shown; Checkout disabled	High	Positive
TC_CART_007	Price	Subtotal with multiple items	1. Add items 2. Open cart	Milk ₹60×2, Bread ₹45×1, Eggs ₹90×3	Subtotal 120+45+270 = ₹435; GST ₹21.75; Delivery ₹40; Total ₹496.75	High	Positive

TC ID	Module	Scenario	Steps	Test Data	Expected Result	Priority	Type
TC_CART_008	Price	Delivery fee boundary – exactly ₹500	Add items totalling ₹500	Subtotal ₹500	Delivery ₹0; GST ₹25.00; Total ₹525.00	High	Boundary
TC_CART_009	Price	Delivery fee boundary – ₹499	Add items totalling ₹499	Subtotal ₹499	Delivery ₹40; GST ₹24.95; Total ₹563.95	High	Boundary
TC_CART_010	Coupon	SAVE10 on ₹1,500 (discount cap)	1. Cart ₹1,500 2. Apply SAVE10	SAVE10	Discount ₹100 (capped, not ₹150); Net ₹1,400; GST ₹70; Delivery ₹0; Total ₹1,470	High	Positive
TC_CART_011	Coupon	SAVE10 below the cap	1. Cart ₹600 2. Apply SAVE10	SAVE10	Discount ₹60; Net ₹540; GST ₹27; Delivery ₹0; Total ₹567	High	Positive
TC_CART_012	Coupon	SAVE10 below minimum order	1. Cart ₹280 2. Apply SAVE10	SAVE10	Error "Minimum order of ₹300 required"; no discount; Total ₹280 + GST ₹14 + Delivery ₹40 = ₹334	High	Negative
TC_CART_013	Coupon	FLAT50 pushes net below free-delivery limit	1. Cart ₹520 2. Apply FLAT50	FLAT50	Net ₹470 (< ₹500) so Delivery ₹40; GST ₹23.50; Total ₹533.50	High	Boundary
TC_CART_014	Coupon	Apply an expired coupon	1. Enter OLD20 2. Click Apply	OLD20	Error "This coupon has expired"; totals unchanged	Medium	Negative
TC_CART_015	Coupon	Apply a second coupon	1. SAVE10 applied 2. Apply FLAT50	SAVE10 + FLAT50	Message "Only one coupon per order"; SAVE10 stays applied; FLAT50 not added	Medium	Negative
TC_CART_016	Coupon	Cart falls below coupon minimum after removal	1. Cart ₹600 + SAVE10 2. Remove items to ₹250	SAVE10	SAVE10 auto-removed with message; totals recalculated: ₹250 + GST ₹12.50 + Delivery ₹40 = ₹302.50	High	Negative
TC_CART_017	Stock	Item goes out of stock while in cart	1. Add Eggs 2. Admin sets Eggs stock 0 3. Refresh cart	Eggs	"Out of stock" label on Eggs; Checkout blocked with message until item removed	High	Negative
TC_CART_018	Persistence	Logged-in cart persists across sessions	1. Login, add 2 items 2. Logout 3. Login on another device	Registered user	Same 2 items with same quantities shown	Medium	Positive
TC_CART_019	Persistence	Guest cart merges on login	1. As guest add Bread 2. Login (user cart has Milk)	Guest + user carts	Cart shows Bread and Milk; no items lost; totals recalculated	Medium	Positive

Observations

OBS	What improved and why
OBS 1	Depth of validation: the weak prompt gave only 8 one-line checks with vague results like "Correct total is displayed". The ICEPOT output gave 19 cases where every price case shows the exact calculation (subtotal, discount, GST, delivery, total), so a tester can actually pass or fail it. This improved because the Context gave the real pricing and coupon rules.
OBS 2	Edge cases appeared that nobody asked for directly: ₹499 vs ₹500 delivery boundary, the SAVE10 ₹100 cap, FLAT50 pulling the net total below ₹500, and a coupon being auto-removed when the cart shrinks. These are the areas where real cart bugs occur, and the AI found them only because the rules were stated.
OBS 3	Consistency and usability: the Example and Output Format made every case follow the same structure (ID, Module, Steps, Test Data, Priority, Type), so the output can be pasted straight into a test management sheet. The weak output had no IDs, test data or priorities and would need to be rewritten before use.

Scenario 04 — Role-Based Access Control

Prompt Makeover Template

Template Field	My Entry
Scenario Name	Role-Based Access Control
Weak Prompt	"Write test cases for user roles."
What I Feel is Missing	1. No application domain or module (ERP, HR, banking?). 2. No role types defined (Admin, Manager, Viewer). 3. No permission matrix showing which role can do which action. 4. No unauthorized access scenarios: hidden buttons, direct URL access, direct API calls. 5. No rules for permission changes or deactivated users. 6. No expected error behaviour (403 page, audit log) and no output format.
I – Instruction	Generate RBAC test cases that verify each role can perform only its permitted actions, and that unauthorized attempts through the UI, direct URL and API are blocked.
C – Context	Domain: ERP accounting application, Invoice module. Roles: Admin – Create, View, Edit, Delete, Approve invoices and manage users/roles. Manager – Create, View, Edit, Approve invoices; cannot Delete or manage users. Viewer – View only. Unauthorized actions must be hidden in the UI; direct URL or API access must return "403 – Access Denied" and be recorded in the audit log. Permission changes made by Admin take effect on the user's next login. Deactivated users cannot log in. Unauthenticated users are redirected to the login page.
E – Example	TC_RBAC_011 Viewer Edit invoice via direct URL Pre-condition: logged in as Viewer Steps: paste /invoice/edit/101 in the address bar Expected: 403 Access Denied page; invoice not editable; attempt recorded in audit log Priority: High Type: Negative/Security
P – Persona	Act as a QA Lead specialising in access-control and security testing of ERP and SaaS applications.
O – Output Format	First, a Permission Matrix table (Roles × Actions, with ✓/✗). Then a test case table with columns: Test Case ID, Role, Action, Access Method (UI/URL/API), Steps, Expected Result, Priority, Type. Cover every cell of the matrix. Minimum 15 test cases.
T – Tone	Structured, precise and security-focused.

☒ Rewritten Prompt

You are a QA Lead specialising in access-control and security testing of ERP and SaaS applications.

Task: Generate role-based access control (RBAC) test cases for the Invoice module of an ERP accounting application. Verify that each role can perform only its permitted actions and that unauthorized attempts through the UI, direct URL and API are blocked.

Roles and rules: Admin can Create, View, Edit, Delete and Approve invoices and manage users/roles. Manager can Create, View, Edit and Approve invoices but cannot Delete or manage users. Viewer can only View. Unauthorized actions must be hidden in the UI; direct URL or API access must return "403 – Access Denied" and be

recorded in the audit log. Permission changes made by Admin take effect on the user's next login. Deactivated users cannot log in. Unauthenticated users are redirected to the login page.

Example test case: TC_RBAC_011 | Viewer | Edit invoice via direct URL | Pre-condition: logged in as Viewer | Steps: paste /invoice/edit/101 in the address bar | Expected: 403 Access Denied page; invoice not editable; attempt recorded in audit log | Priority: High | Type: Negative/Security.

Output: First a Permission Matrix (Roles × Actions with ✓/✗). Then a test case table with columns Test Case ID, Role, Action, Access Method (UI/URL/API), Steps, Expected Result, Priority, Type. Cover every cell of the matrix. Minimum 15 test cases.

Tone: Structured, precise and security-focused.

Output 1 — Weak Prompt ("Write test cases for user roles.")

#	Test Case	Expected Result
1	Verify admin can log in successfully	Admin dashboard is displayed
2	Verify user can log in with an assigned role	User is logged in
3	Verify admin can create a new user	User is created
4	Verify admin can assign a role to a user	Role is assigned
5	Verify normal user cannot access the admin page	Access is denied
6	Verify roles are displayed correctly in the list	Roles are listed
7	Verify user can log out	User is logged out

Result: 7 generic checks, no IDs, no test data, no priority, no domain rules.

Output 2 — ICEPOT Prompt

Permission Matrix

Role	View Invoice	Create Invoice	Edit Invoice	Delete Invoice	Approve Invoice	Manage Users/Roles
Admin	✓	✓	✓	✓	✓	✓
Manager	✓	✓	✓	X	✓	X
Viewer	✓	X	X	X	X	X

Test Cases

TC ID	Role	Action	Method	Steps	Expected Result	Priority	Type
TC_RBAC_001	Admin	Create invoice	UI	1. Login as Admin 2. Invoices → New 3. Fill details, Save	Invoice saved with a new number; appears in list	High	Positive
TC_RBAC_002	Admin	Delete invoice	UI	1. Open INV-101 2. Click Delete 3. Confirm	Confirmation shown; invoice deleted; action in audit log	High	Positive

TC ID	Role	Action	Method	Steps	Expected Result	Priority	Type
TC_RBAC_003	Admin	Manage users/roles	UI	1. Settings → Manage Users 2. Change a user's role	Page accessible; role change saved	High	Positive
TC_RBAC_004	Manager	Create and edit invoice	UI	1. Login as Manager 2. Create invoice 3. Edit and save	Both actions succeed	High	Positive
TC_RBAC_005	Manager	Approve invoice	UI	1. Open pending INV-102 2. Click Approve	Status changes to Approved	High	Positive
TC_RBAC_006	Manager	Delete invoice	UI	1. Open INV-101 2. Look for Delete option	Delete button not visible in detail page or list actions	High	Negative
TC_RBAC_007	Manager	Manage users	URL	Paste /settings/users in address bar	403 Access Denied page; attempt logged in audit trail	High	Negative/Security
TC_RBAC_008	Manager	Delete invoice	API	Send DELETE /api/invoices/101 with Manager token	403 response; invoice still exists	High	Negative/Security
TC_RBAC_009	Viewer	View invoice	UI	1. Login as Viewer 2. Open INV-101	Invoice details visible in read-only mode	High	Positive
TC_RBAC_010	Viewer	Create/Edit/Delete/Approve	UI	Check invoice list and detail page for action buttons	New, Edit, Delete and Approve buttons all hidden	High	Negative
TC_RBAC_011	Viewer	Edit invoice	URL	Paste /invoice/edit/101 in address bar	403 Access Denied; invoice not editable; audit log entry	High	Negative/Security
TC_RBAC_012	Viewer	Create invoice	API	Send POST /api/invoices with Viewer token	403 response; no invoice created	High	Negative/Security
TC_RBAC_013	Viewer	Approve invoice (tampered request)	API	Send approve request with role=Admin added to the body	Server ignores client role; 403 response; status unchanged	Medium	Negative/Security
TC_RBAC_014	Admin → Manager	Revoke Approve permission	UI	1. Admin removes Approve from Manager 2. Manager logs out and in	Approve button hidden for Manager after re-login	High	Positive
TC_RBAC_015	Manager	Permission change during active session	UI	1. Manager logged in 2. Admin revokes Approve 3. Manager continues without re-login	Change applies only on next login, as per rule; verify behaviour matches and is documented	Medium	Boundary
TC_RBAC_016	Any	Deactivated user login	UI	1. Admin deactivates user 2. User tries to log in	Login blocked with "Account is inactive" message	High	Negative

TC ID	Role	Action	Method	Steps	Expected Result	Priority	Type
TC_RBAC_017	None	Unauthenticated access	URL	Open /invoice/101 without logging in	Redirected to login page; no invoice data shown	High	Negative/Security

Observations

OBS	What improved and why
OBS 1	The weak prompt did not test roles at all — it mostly tested login, logout and user creation. The ICEPOT output started with a permission matrix and then covered every role-action combination, so it is easy to prove that nothing was missed. The Context (role list and rules) made this possible.
OBS 2	Security coverage improved a lot. The weak output had one vague line "normal user cannot access admin page". The ICEPOT output tests the same restriction at three levels — hidden button (UI), direct URL and direct API call — plus a tampered request. Real RBAC defects are usually found at the URL/API level, not in the UI.
OBS 3	Expected results became measurable: "403 Access Denied", "audit log entry", "button not visible", "redirected to login". The weak output's "Access is denied" could not be verified the same way by two different testers. The Persona (QA Lead, security-focused) and the Example set this standard.

Scenario 05 — API Testing

Prompt Makeover Template

Template Field	My Entry
Scenario Name	API Testing
Weak Prompt	"Write API test cases for user registration."
What I Feel is Missing	1. No endpoint URL or HTTP method (POST). 2. No request headers or request body structure (JSON fields). 3. No field validation rules (mandatory fields, email format, password policy, phone length). 4. No status codes to validate (201, 400, 409, 415, 405). 5. No response structure or response-time expectation. 6. No tool context (Postman / RestAssured) and no output format.
I – Instruction	Write API test cases for the user registration endpoint covering valid requests, field validations, duplicate users, invalid payloads, headers, HTTP methods, status codes, response schema and response time.
C – Context	Endpoint: POST https://api.example.com/v1/users/register. Header: Content-Type: application/json. Request body: { "firstName": "Priya", "lastName": "K", "email": "priya@test.com", "password": "Test@1234", "phone": "9876543210" }. Rules: all fields mandatory; email valid and unique; password 8–16 characters with at least 1 uppercase, 1 number and 1 special character; phone exactly 10 digits. Responses: 201 Created with { userId, email, createdAt } (password never returned); 400 Bad Request with an error per field; 409 Conflict for duplicate email; 415 for wrong Content-Type; 405 for wrong method. Response time < 1000 ms. Tool: Postman with test scripts; RestAssured automation later.
E – Example	TC_API_002 Register with an existing email POST Body: email = priya@test.com (already registered) Expected: 409, { "error": "Email already registered" }, response time < 1000 ms Postman: pm.response.to.have.status(409)
P – Persona	Act as an API Test Engineer experienced with Postman, RestAssured and REST standards.
O – Output Format	A table with columns: Test Case ID, Scenario, Method, Request Body / Headers, Expected Status, Expected Response & Validation Points. After the table, give sample Postman test scripts for 3 key cases. Minimum 15 test cases.
T – Tone	Technical, concise and developer-friendly.

☒ Rewritten Prompt

You are an API Test Engineer experienced with Postman, RestAssured and REST standards.

Task: Write API test cases for the user registration endpoint covering valid requests, field validations, duplicate users, invalid payloads, headers, HTTP methods, status codes, response schema and response time.

API details: POST https://api.example.com/v1/users/register, Header Content-Type: application/json. Request body: { "firstName": "Priya", "lastName": "K", "email": "priya@test.com", "password": "Test@1234", "phone": "9876543210" }. All fields are mandatory; email must be valid and unique; password 8–16 characters with at least 1 uppercase, 1 number and 1 special character; phone exactly 10 digits. Responses: 201 Created with { userId, email, createdAt } and the password never returned; 400 Bad Request with an error per field; 409 Conflict for duplicate email; 415 for wrong Content-Type; 405 for wrong method. Response time must be under 1000 ms. Tool: Postman with test scripts; RestAssured automation will follow.

Example test case: TC_API_002 | Register with an existing email | POST | Body: email = priya@test.com (already registered) | Expected: 409, { "error": "Email already registered" }, response time < 1000 ms | Postman: pm.response.to.have.status(409).

Output: A table with columns Test Case ID, Scenario, Method, Request Body / Headers, Expected Status, Expected Response & Validation Points. After the table, give sample Postman test scripts for 3 key cases. Minimum 15 test cases.

Tone: Technical, concise and developer-friendly.

Output 1 — Weak Prompt ("Write API test cases for user registration.")

#	Test Case	Expected Result
1	Verify user registration with valid data	User is registered successfully
2	Verify registration with an invalid email	Error message is displayed
3	Verify registration with empty fields	Validation error is shown
4	Verify registration with an already existing user	Error is returned
5	Verify password validation	Password rules are enforced
6	Verify success message is displayed after registration	Success message appears

Result: 6 generic checks, no IDs, no test data, no priority, no domain rules.

Output 2 — ICEPOT Prompt

TC ID	Scenario	Method	Request Body / Headers	Status	Expected Response & Validation Points
TC_API_001	Register with all valid fields	POST	Valid body as in spec; Content-Type: application/json	201	Body has userId, email, createdAt; email matches request; password NOT in response; time < 1000 ms
TC_API_002	Duplicate email	POST	email = priya@test.com (already exists)	409	{ "error": "Email already registered" }; no new user created
TC_API_003	Missing mandatory field	POST	firstName removed from body	400	Error for firstName: "firstName is required"
TC_API_004	Empty body	POST	{ }	400	Errors returned for all 5 fields
TC_API_005	Invalid email format	POST	email = "priyatest.com"	400	Error for email: invalid format
TC_API_006	Password below minimum (7 chars)	POST	password = "Test@12"	400	Error for password: length must be 8–16
TC_API_007	Password at maximum (16 chars)	POST	password = "Test@12345678901"	201	User created; boundary value accepted
TC_API_008	Password above maximum (17 chars)	POST	password = "Test@123456789012"	400	Error for password length
TC_API_009	Password without special character	POST	password = "Test12345"	400	Error: must contain a special character
TC_API_010	Phone with 9 digits	POST	phone = "987654321"	400	Error for phone: must be 10 digits
TC_API_011	Phone with letters	POST	phone = "98765abcde"	400	Error for phone: digits only
TC_API_012	Wrong Content-Type	POST	Content-Type: text/plain	415	Unsupported Media Type; no user created

TC ID	Scenario	Method	Request Body / Headers	Status	Expected Response & Validation Points
TC_API_013	Wrong HTTP method	GET	No body	405	Method Not Allowed
TC_API_014	Malformed JSON	POST	Body with missing closing brace	400	Parse error message; no 500 error; no stack trace exposed
TC_API_015	SQL injection in name	POST	firstName = "" OR 1=1--"	400 / 201	No 500 error; input rejected or stored as plain text; no DB error in response

Sample Postman Test Scripts

TC_API_001 — Valid registration

```
pm.test("Status is 201", () => pm.response.to.have.status(201));
pm.test("Response time < 1000 ms", () => pm.expect(pm.response.responseTime).to.be.below(1000));
const res = pm.response.json();
pm.test("Has userId, email, createdAt", () => {
  pm.expect(res).to.have.property("userId");
  pm.expect(res.email).to.eql("priya@test.com");
  pm.expect(res).to.have.property("createdAt");
});
pm.test("Password not returned", () => pm.expect(res).to.not.have.property("password"));
```

TC_API_002 — Duplicate email

```
pm.test("Status is 409", () => pm.response.to.have.status(409));
pm.test("Error message", () => pm.expect(pm.response.json().error).to.eql("Email already registered"));
```

TC_API_006 — Short password

```
pm.test("Status is 400", () => pm.response.to.have.status(400));
pm.test("Password error present", () => pm.expect(JSON.stringify(pm.response.json())).to.include("password"));
```

Observations

OBS	What improved and why
OBS 1	The weak output read like UI test cases ("Success message appears", "Error message is displayed") and did not mention a single HTTP method or status code. The ICEPOT output specifies the method, exact request body and expected status code (201, 400, 409, 415, 405) for every case. The Context supplied the endpoint and response rules that made this possible.
OBS 2	Boundary and negative coverage became systematic: password lengths 7 / 16 / 17, 9-digit and alphabetic phone numbers, empty body, malformed JSON, wrong Content-Type, wrong method and SQL injection. The weak output had one generic line "Verify password validation" that could mean anything.

OBS	What improved and why
OBS 3	Because the tool (Postman) was given in the prompt, the output included ready-to-run Postman test scripts that check status, response time, response fields and that the password is not returned. This moves the output from a checklist to something directly executable and a good base for RestAssured automation later.

Summary — Weak vs ICEPOT Output

Scenario	Weak: test cases	ICEPOT: test cases	ICEPOT: negative / boundary / security cases	Key additions from ICEPOT
03 — Shopping Cart	8	19	11	Exact price calculations, GST, delivery boundary, coupon caps/minimums, out-of-stock, guest cart merge
04 — RBAC	7	17 + permission matrix	10	Role × action matrix, UI/URL/API-level checks, 403 + audit log, permission change timing, deactivated users
05 — API Testing	6	15 + 3 Postman scripts	13	Methods, request bodies, status codes, schema checks, boundary values, security, runnable scripts

Overall Learning

- Context has the biggest impact: the AI cannot test rules it has not been told. Every boundary case in the ICEPOT outputs came directly from a rule written in the Context.
- The Example sets the quality bar: the AI copied the ID format, level of detail and calculation style from the single example given.
- Persona and Output Format turn a checklist into a usable deliverable: consistent columns, priorities and types mean the output can go straight into a test execution sheet.
- The weak prompt is still useful as a starting point for brainstorming, but it should never be used as final test documentation.